# Data Wrangling Tidyverse

Kathryn Gour

2025-10-16

## Exercises

The following exercises will test your ability to apply the data wrangling skills you've learned in this chapter. Each problem uses datasets that are built into the Tidyverse, so you will need to have the `tidyverse` library loaded. The goal is not just to get the right answer, but to write a clean, readable pipeline of commands for each task. **Don't forget to use text blocks and/or comments to document your thinking and explain what each line of your code is doing.**

---

**Problem 1: Analyzing US Arrests**

**Dataset:** `USArrests` (built into R)

**Scenario:** A law student at William & Mary is researching historical crime data from 1973 and wants to identify states with high urban populations and their corresponding violent crime rates.

**Task:** Write a single pipeline of `dplyr` commands that: 1. Starts with the `USArrests` dataset. Note that the state names are currently row names. You can turn them into a proper column by running `USArrests %>% rownames_to_column(var = "state")` first. 2. Filters the data to include only states with an `UrbanPop` rate of 80% or higher. 3. Selects only the `state`, `Murder`, and `Rape` columns. 4. Arranges the final result by the `Murder` rate in descending order. 5. Write a sentence or two to describe the result.

```r
# Your code here
#let's transpose state names to a column, filter the rows displayed by
#UrbanPop>80, and select states that have Murder and Rape arrests.
#Then we can arrange the dataset by descending Murder rates.
USArrests %>%
  rownames_to_column(var = "state") %>%
  filter(UrbanPop > 80) %>%
  select(state, Murder, Rape) %>%
  arrange(desc(Murder))
```

```
##           state Murder Rape
## 1        Nevada   12.2 46.0
## 2      New York   11.1 26.1
## 3      Illinois   10.4 24.0
## 4    California    9.0 40.6
## 5    New Jersey    7.4 18.8
## 6        Hawaii    5.3 20.2
## 7 Massachusetts    4.4 16.3
## 8  Rhode Island    3.4  8.3
```

**Describe the result:** Through our data filtering and reshaping, we can see that Nevada, New York, and Illinois are the US states that have the highest rates of murder arrests.

---

**Problem 2: Mammal Sleep Habits**

**Dataset:** `msleep` (from the `ggplot2` package)

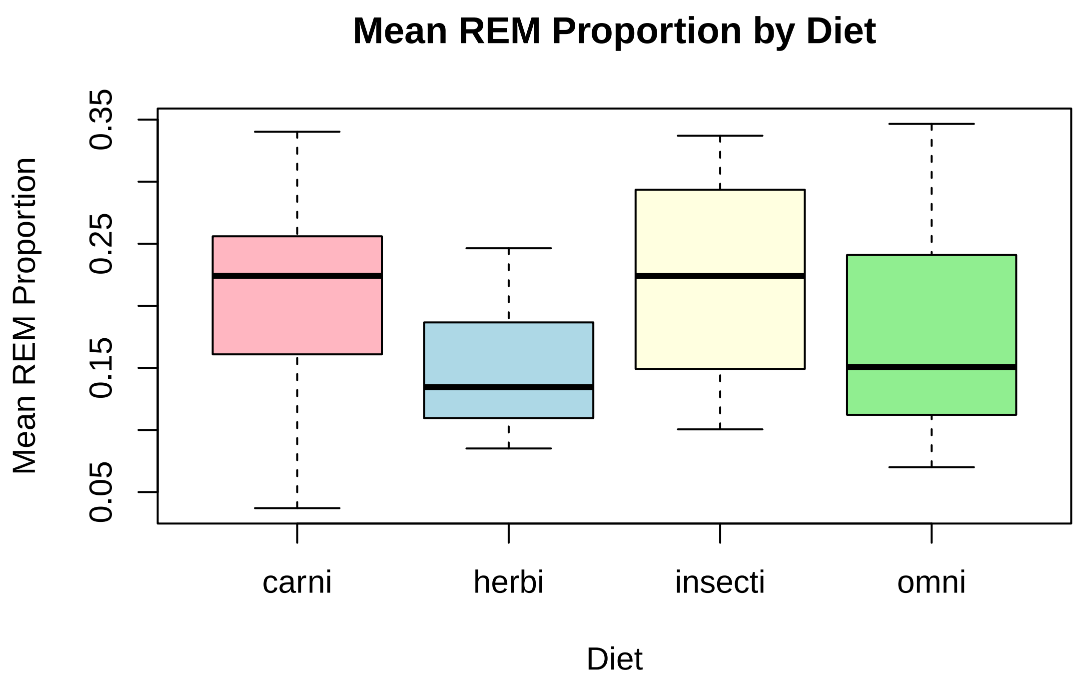
**Scenario:** A researcher at the Virginia Living Museum wants to know if there's a relationship between an animal's diet and its sleep patterns, specifically the proportion of time spent in REM sleep.

**Task:** Write a `dplyr` pipeline that: 1. Starts with the `msleep` dataset. 2. Removes rows where the `vore` (diet) column is `NA`, as we can't use them for this analysis. (Hint: use `filter(!is.na(vore))`). 3. Creates a new column called `rem_proportion` which is the ratio of `sleep_rem` to `sleep_total`. 4. Groups the data by `vore`. 5. Calculates the average `rem_proportion` for each diet group. 6. Arranges the final tibble to show the diets with the highest average REM proportion first. 7. Create a bar chart or boxplot to visualize the REM proportion for each diet type. 8. Write a sentence or two to describe the results.

```r
# Your code here
msleep %>%
  filter(!is.na(vore)) %>% #filter out NAs
  #we can use mutate to make a new column that ratios sleep_rem and sleep_total
  mutate(rem_proportion = sleep_rem/sleep_total) %>%
  #we can use groupby to arrange the data by vore
  group_by(vore) %>%
  #we can use summarise to calculate the average of rem_proportion
  summarise(mean_rem_proportion = mean(rem_proportion, na.rm = TRUE)) %>%
  #show descending order of mean REM proportion
  arrange(desc(mean_rem_proportion))
```

```
## # A tibble: 4 x 2
##   vore    mean_rem_proportion
##   <chr>                 <dbl>
## 1 insecti               0.221
## 2 carni                 0.207
## 3 omni                  0.179
## 4 herbi                 0.149
```

```r
  #now we can make a box plot of the wrangled data!
  boxplot((sleep_rem/sleep_total) ~ vore,
        data = msleep,
        main = "Mean REM Proportion by Diet",
        xlab = "Diet",
        ylab = "Mean REM Proportion",
        col = c("lightpink","lightblue","lightyellow","lightgreen"))
```




**Describe the result:** The wrangled msleep data shows that herbivores and omnivores have the lowest REM proportion. —

### Problem 3: Tidying Global Health Data

**Dataset:** `who` (from the `tidyr` package)

**Scenario:** An intern at the Virginia Department of Health has been given a dataset from the World Health Organization about tuberculosis cases. The data is in an extremely "wide" and messy format and needs to be tidied before any analysis can be done.

**Task:** This is a classic reshaping challenge. The columns from `new_sp_m014` through `newrel_f65` contain values that should be in their own columns. 1. Start with the `who` dataset. 2. Use `pivot_longer()` to gather all the columns from `new_sp_m014` to `newrel_f65`. 3. The names of these columns should go into a new column called `diagnosis_code`. 4. The values should go into a new column called `count`. 5. After pivoting, many rows will have a `count` of `NA`. Filter these rows out. 6. Display the first few rows of your final tidy tibble.

*(Hint: This is a complex pivot! You can select the columns with `cols = new_sp_m014:newrel_f65`)*

```r
# Your code here
who_wrangled <- who %>%
  pivot_longer(
    cols = new_sp_m014:newrel_f65,    # get all tuberculosis case columns
    names_to = "diagnosis_code",      # put column names into this new column
    values_to = "count"               # put cases into this new column
  ) %>%
  filter(!is.na(count))               # remove NAs

# show first few rows of the tibble
head(who_wrangled)
```

```
## # A tibble: 6 x 6
##   country     iso2  iso3   year diagnosis_code count
##   <chr>       <chr> <chr> <dbl> <chr>          <dbl>
```

```
## 1 Afghanistan AF    AFG    1997 new_sp_m014        0
## 2 Afghanistan AF    AFG    1997 new_sp_m1524      10
## 3 Afghanistan AF    AFG    1997 new_sp_m2534       6
## 4 Afghanistan AF    AFG    1997 new_sp_m3544       3
## 5 Afghanistan AF    AFG    1997 new_sp_m4554       5
## 6 Afghanistan AF    AFG    1997 new_sp_m5564       2
```

**Describe the Result:** Now we can see that Afghanistan in 1997 had multiple diagnoses of tuberculosis. —

### Problem 4: Finding the Windiest Year for Storms

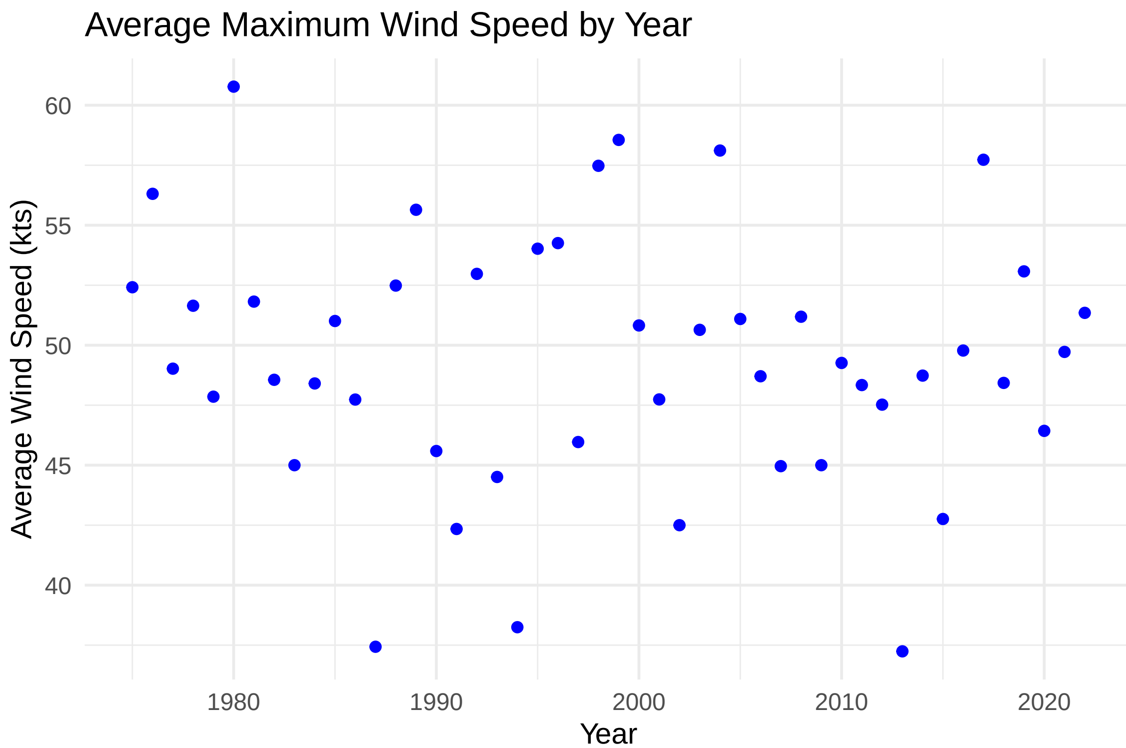
**Dataset:** `storms` (from the `dplyr` package)

**Scenario:** Meteorologists in Hampton Roads are analyzing the historical Atlantic storm dataset to identify the year with the highest average maximum wind speeds, which could indicate a particularly active storm season.

**Task:** Write a `dplyr` pipeline to determine which year had the highest average wind speeds for its storms. Your final output should be a tibble with only one row (the winning year). 1. Start with the `storms` dataset. 2. Group the data by `year`. 3. For each year, calculate the average maximum wind speed (`wind`). 4. Arrange the results to find the year with the highest average. 5. Create a scatter plot of year as a function of wind speed. 6. Return only the top row. (Hint: `head(1)` or `slice_max()` are good ways to do this). 7. Write a sentence or two to describe your results.

```r
# Your code here
storms_wrangled <- storms %>%
  group_by(year) %>%
  # calculate mean wind speed
  summarise(mean_wind = mean(wind, na.rm = TRUE)) %>%
  # mean wind speed in descending order
  arrange(desc(mean_wind))

# scatter plot of year against mean wind speed
ggplot(storms_wrangled, aes(x = year, y = mean_wind)) +
  geom_point(color = "blue") +
  labs(
    title = "Average Maximum Wind Speed by Year",
    x = "Year",
    y = "Average Wind Speed (kts)"
    #fun fact i didn't know wind speed was measured in kts until this assignment
  ) +
  theme_minimal()
```




```r
# find the year with the highest average wind speed
top_year <- storms_wrangled %>%
  slice_max(mean_wind, n = 1)

top_year
```

```
## # A tibble: 1 x 2
##    year mean_wind
##   <dbl>     <dbl>
## 1  1980      60.8
```

**Describe the Result:** Now we can see from the scatterplot that there seems to be no noticeable trend of average wind speed across year (AKA there's no positive or negative trends). Furthermore, from the tibble, we can see that the highest average wind speed in 1980 was 60.8 knots! —

**Problem 5: From Tidy back to Wide**

**Dataset:** Your tidied data from Problem 3.

**Scenario:** After you tidied the `who` dataset, your manager asks for a specific summary table: a report showing the total number of cases for Afghanistan, Brazil, and China for the years 1999 through 2001. They want the years as columns for a quick comparison.

**Task:** This requires you to filter, summarize, and then reshape your tidy data back into a wide format. 1. Start with the tidied data you created in Problem 3. 2. `filter()` for only the countries of interest (`Afghanistan`, `Brazil`, `China`) and the years of interest (`1999`, `2000`, `2001`). 3. Calculate the total number of cases (`count`) for each country and year combination. (Hint: `group_by()` then `summarise()`). 4. Use `pivot_wider()` to reshape this summary table. The new column headers should come from the `year` column, and the values should come from your new `total_cases` column. 5. Write a sentence or two to describe your results.

```r
# First, recreate your tidy 'who' data from Problem 3
tidy_who <- who %>%
```

```r
  pivot_longer(
    cols = new_sp_m014:newrel_f65,      # all the TB case columns
    names_to = c("diagnosis", "gender", "age"),
    names_pattern = "new_?(.*)_(.)(.*)",
    values_to = "cases"
  ) %>%
  # replace missing diagnosis with "sp"
  mutate(
    diagnosis = ifelse(diagnosis == "", "sp", diagnosis)
  ) %>%
  select(country, year, diagnosis, gender, age, cases)

# Then, write the pipeline for this problem
tidy_who %>%
  filter(
    country %in% c("Afghanistan", "Brazil", "China"),
    year %in% c(1999, 2000, 2001)
  ) %>%
  group_by(country, year) %>%
  summarise(total_cases = sum(cases, na.rm = TRUE)) %>%
  pivot_wider(
    names_from = year,
    values_from = total_cases
  )
```

```
## `summarise()` has grouped output by 'country'. You can override using the
## `.groups` argument.
```

```
## # A tibble: 3 x 4
## # Groups:   country [3]
##   country     `1999` `2000` `2001`
##   <chr>        <dbl>  <dbl>  <dbl>
## 1 Afghanistan    745   2666   4639
## 2 Brazil       37737  80488  37491
## 3 China       212258 213766 212766
```

**Describe the Result:** Now we can see from the tibble that in Afghanistan, Brazil, and China there was an increase in TB cases from the years 1999-2000, but a decrease in cases in 2001 (except for in Afghanistan). —

### Problem 6: Ranking Species by Average BMI (Star Wars)

**Dataset:** starwars (from dplyr)

**Scenario:** A sci-fi physiology class wants to compare species by average Body Mass Index (BMI), computed from height and mass.

#### Task:

Write a pipeline that:

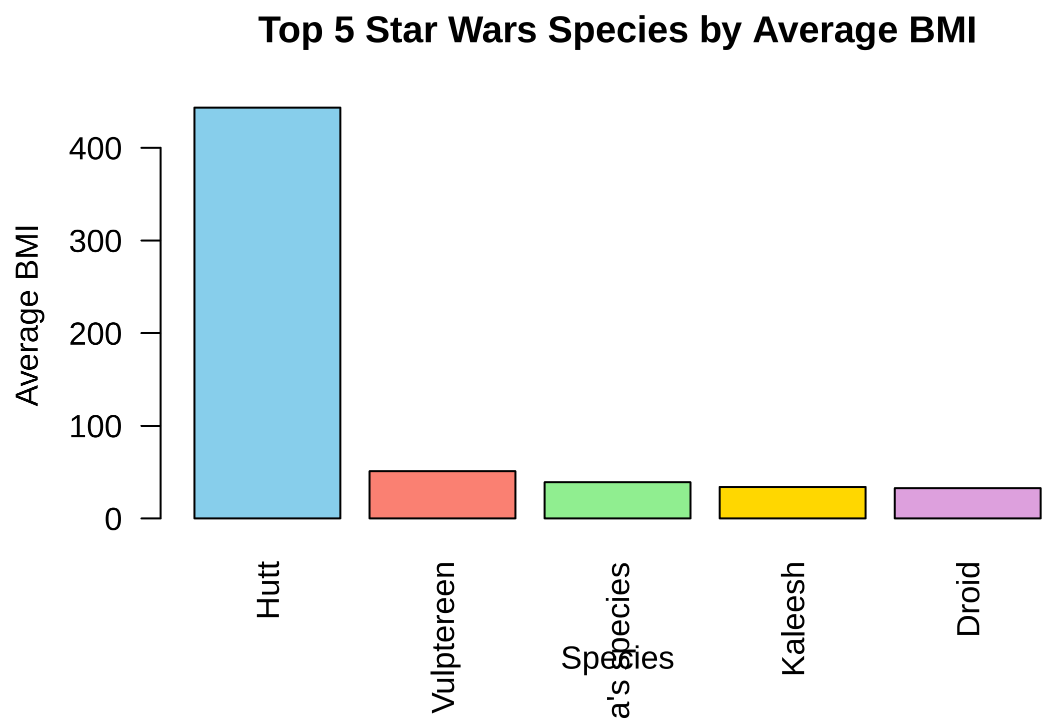
1. Starts from starwars, and keeps name, species, height, mass.
2. Creates height_m = height / 100 and bmi = mass / (height_m^2).
3. Groups by species, computes avg_bmi and n (characters per species).
4. Ungroup and return the top 5 species by avg_bmi, breaking ties by n (larger first).

5. Create a visualization (plot) of your results.
6. Write a sentence or two to describe your results.

(Tip: This reinforces mutate(), summarise(), arrange(), and the importance of ungroup().)

```
# Your code here
starwars_wrangled <- starwars %>%
  select(name, species, height, mass) %>%
  # create height_m and bmi from the formulas above
  mutate(height_m = height/100, bmi = mass/(height_m^2)) %>%
  # group by species
  group_by(species) %>%
  summarise(mean_bmi = mean(bmi, na.rm = TRUE),
            n = n()) %>%
  # ungroup
  ungroup() %>%
  arrange(desc(mean_bmi), desc(n)) %>%
  head(5)

# bar plot of species by avg bmi
  barplot(
  height = starwars_wrangled$mean_bmi,
  names.arg = starwars_wrangled$species,
  main = "Top 5 Star Wars Species by Average BMI",
  xlab = "Species",
  ylab = "Average BMI",
  col = c("skyblue", "salmon", "lightgreen", "gold", "plum"),
  las = 2
)
```




**Describe the Result:** Now we can see from the bar plot that the top 5 starwars species by BMI are Hutts, Vulptereen, Yoda's species, Kaleesh, and Droid. –

**Problem 7: Advanced Reshaping with `.value` (Water Quality)**

**Dataset:** Recreate the `water_quality_wide` tibble from the chapter (or copy it in your chunk).

**Scenario:** You need a tidy table that separates pollutant and statistic, then a compact report.

**Task:**
1. Start from `water_quality_wide` (columns like `nitrogen_mean`, `nitrogen_sd`, etc.).
2. Use `pivot_longer()` with `names_to = c("pollutant", ".value")` and `names_sep = "_"` to get columns `site, collection_date, pollutant, mean, sd`.
3. Compute **overall means by pollutant** (average of the `mean` column across sites/dates).
4. Return a **one-row per pollutant** report and then `pivot_wider()` so pollutants are columns and the overall means are values.
5. Write a sentence or two describing your results.

*(Tip: This exercises the advanced `.value` pattern and a round-trip pivot.)*

```r
# Your code here
# A more complex "wide" tibble
water_quality_wide <- tribble(
  ~site,       ~collection_date, ~nitrogen_mean, ~nitrogen_sd, ~phosphorus_mean, ~phosphorus_sd,
  "Jamestown", "2025-10-01",     1.25,           0.15,         0.08,             0.02,
  "Yorktown",  "2025-10-01",     0.95,           0.11,         0.06,             0.01,
  "Jamestown", "2025-10-02",     1.31,           0.12,         0.09,             0.02,
  "Yorktown",  "2025-10-02",     0.99,           0.09,         0.07,             0.01
)

print(water_quality_wide)
```

```
## # A tibble: 4 x 6
##   site   collection_date nitrogen_mean nitrogen_sd phosphorus_mean phosphorus_sd
##   <chr>  <chr>                   <dbl>       <dbl>           <dbl>         <dbl>
## 1 James~ 2025-10-01               1.25        0.15            0.08          0.02
## 2 Yorkt~ 2025-10-01               0.95        0.11            0.06          0.01
## 3 James~ 2025-10-02               1.31        0.12            0.09          0.02
## 4 Yorkt~ 2025-10-02               0.99        0.09            0.07          0.01
```

```r
# reshape wide water_quality to long
water_quality_long <- water_quality_wide %>%
  pivot_longer(
    # we can use '-c' to exclude columns instead of writing everything out
    cols = -c(site, collection_date),
    names_to = c("pollutant", ".value"),
    names_sep = "_" # telling it to filter the names by underscore
  )

# compute overall mean of the `mean` column by pollutant
pollutant_means <- water_quality_long %>%
  group_by(pollutant) %>%
  summarise(overall_mean = mean(mean, na.rm = TRUE)) %>%
  ungroup()

# pivot wider so pollutants become columns
pollutant_means_wide <- pollutant_means %>%
  pivot_wider(
```

```r
    names_from = pollutant,
    values_from = overall_mean
  )

# print it!
pollutant_means_wide
```

```
## # A tibble: 1 x 2
##   nitrogen phosphorus
##      <dbl>      <dbl>
## 1     1.12      0.075
```

**Describe the Result:** Now we can see from the tibble that the overall means of nitrogen and phosphorus pollutant levels in Jamestown and Yorktown VA are 1.125 and 0.075, respectively! –

### Problem 8: Create Your Own Pipeline

**Dataset:** Import your own dataset, or choose from one of the built-in datasets

**Scenario:** Examine the variables and formulate a research question about the data

**Task:** Create a data wrangling pipeline appropriate to your research question, then summarize and plot the results.

```r
# Your code here
# let's use starwars bc best movie series
# We are an intergalatic data analyst interested in investigating the tallest
#species in the universe and what their homeworlds are to find the tallest
#planets! We will do this by comparing height, species, and homeworld.

starwars_postdoc <- starwars %>%
  select(name, species, height, homeworld) %>%
  # remove NAs
  filter(!is.na(species), !is.na(height)) %>%
  # group by both species and homeworld
  group_by(species, homeworld) %>%
  # compute average height
  summarise(
    avg_height = mean(height, na.rm = TRUE),
    n = n()
  ) %>%
  ungroup() %>%
  # sort tallest first
  arrange(desc(avg_height)) %>%
  # keep top 5
  slice_head(n = 5)
```

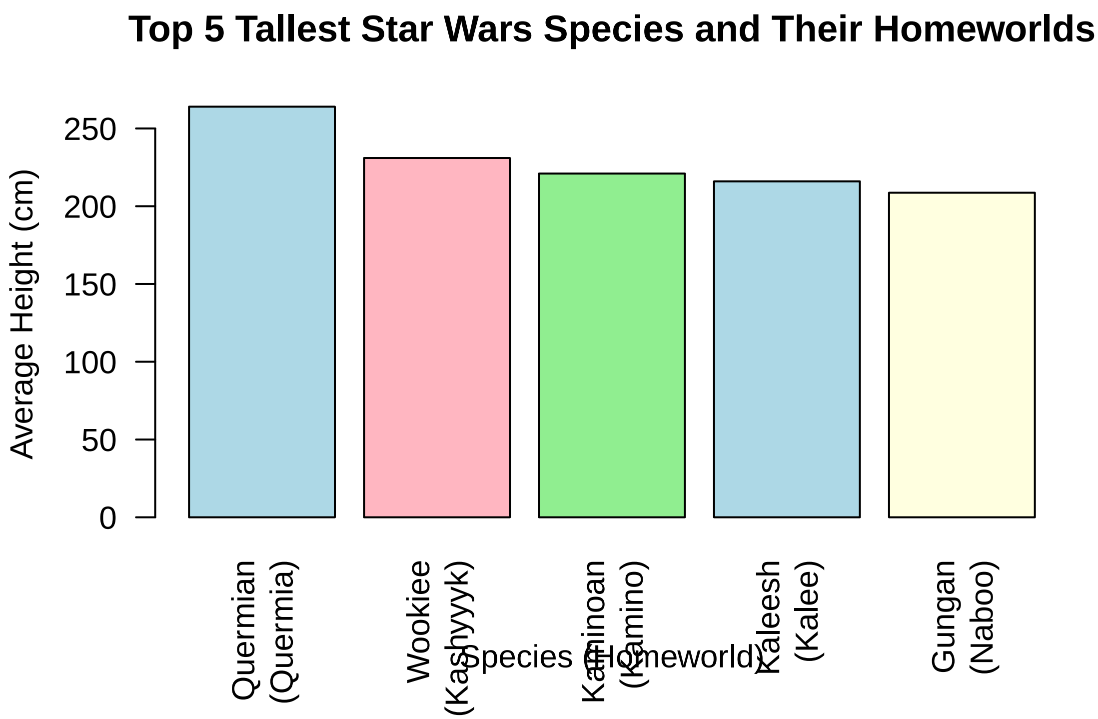
```
## `summarise()` has grouped output by 'species'. You can override using the
## `.groups` argument.
```

```r
starwars_postdoc
```

```
## # A tibble: 5 x 4
##   species  homeworld avg_height     n
##   <chr>    <chr>          <dbl> <int>
## 1 Quermian Quermia          264     1
## 2 Wookiee  Kashyyyk         231     2
```

```
## 3 Kaminoan Kamino          221       2
## 4 Kaleesh  Kalee           216       1
## 5 Gungan   Naboo           209.      3
```

```r
# bar plot of species and homeworld by height
barplot(
  height = starwars_postdoc$avg_height,
  names.arg = paste(starwars_postdoc$species, "\n(", starwars_postdoc$homeworld,
                    ")", sep = ""),
  main = "Top 5 Tallest Star Wars Species and Their Homeworlds",
  xlab = "Species (Homeworld)",
  ylab = "Average Height (cm)",
  col = c("lightblue", "lightpink", "lightgreen", "lightblue", "lightyellow"),
  las = 2
)
```




**Describe the Result:** From our bar plot, we can see that the tallest species are the Quermians from Quermia!